

Mobile Application Testing

iOS • Android • Responsive Web • Touch • Performance

Brian Scott McCarthy | Senior QA Lead | QA Best Practice Series — BP-04

1. Mobile Testing Strategy

Mobile testing must address dimensions unique to mobile environments: touch interaction, device fragmentation, network variability, battery/memory constraints, OS version differences, and platform-specific behaviours (iOS vs Android). Never assume desktop test results apply to mobile.

2. Device & OS Coverage Matrix

Platform	Priority Devices	OS Versions	Browser
iOS — Phone	iPhone 15, iPhone 14, iPhone SE (small screen)	iOS 17, iOS 16	Safari (primary), Chrome
iOS — Tablet	iPad Pro 12.9", iPad Air	iPadOS 17, 16	Safari, Chrome
Android — Phone	Google Pixel 8, Samsung Galaxy S24, Samsung A54 (mid-range)	Android 14, 13, 12	Chrome (primary), Samsung Internet
Android — Tablet	Samsung Galaxy Tab S9, Google Pixel Tablet	Android 14, 13	Chrome
Emulation (CI)	Playwright mobile emulation: iPhone 14, Pixel 7, Galaxy S21	Latest	Chromium/WebKit
Real Device Cloud	BrowserStack or Sauce Labs for P1 device/OS combos	As above	Native browsers

Rule: Always validate P1 scenarios on at least one real iOS device and one real Android device before release. Emulation catches most issues but not all — especially touch events, camera, and native OS integrations.

3. Touch & Gesture Testing

What to Test	
All buttons, links, and interactive elements respond to tap — no click-only event listeners (iOS Safari issue)	
Context menus and long-press actions trigger correctly; no accidental triggering on short tap	
Carousels, tabs, and swipe-to-dismiss respond smoothly; no accidental swipe on scroll	
Page scrolls smoothly without triggering horizontal swipe; pull-to-refresh works if implemented	
If zoom is disabled (meta viewport user-scalable=no), verify accessibility justification; if enabled, content reflows	
No accidental zoom on double-tap for interactive elements; if double-tap is a feature, test it	
Drag handles are large enough (min 44x44px touch target); drop zones highlight on hover	
All tappable elements meet WCAG minimum 44x44px touch target size	

4. Mobile-Specific Functional Tests

Test Area	Key Scenarios
Orientation change	Rotate device mid-session: layout adapts; no data loss; modal/overlay stays in place
Keyboard behaviour	Virtual keyboard appears for correct input types (number, email, tel); layout does not break when keyboard open
Auto-fill & autofill hints	Email and password fields have correct autocomplete attributes; browser autofill works
Deep linking	App/page opens correctly from email links, push notifications, QR codes, and social share links
Offline / poor network	Graceful degradation on 2G/3G; cached content available offline where expected; retry logic on reconnect
App interruptions	Incoming call mid-session; push notification during checkout; switching apps and returning — state preserved
Location services	Permission prompt handled gracefully; denial results in fallback (manual address entry, not crash)
Camera / media access	Permission prompt before first use; denial handled gracefully; media upload works via photo library
Push notifications	Received when app in background; tapping notification deep-links to correct screen
Native share	Share functionality opens system share sheet on both iOS and Android

5. Responsive Web Testing

Breakpoint	Target Devices	Key Layout Checks
320px – 375px	iPhone SE, small Android phones	Nothing overflows viewport; navigation collapses to hamburger; text readable without zoom
376px – 414px	iPhone 14, standard Android	Two-column grids readable; CTAs full-width; images scale correctly
415px – 768px	Large phones, small tablets	Comfortable touch targets; forms not cramped; sidebars collapse
769px – 1024px	iPad portrait, large tablets	Navigation may shift to sidebar; content can use 2–3 columns
1025px – 1440px	iPad landscape, laptops	Full desktop layout; hover states active
1441px+	Large monitors	Max-width container prevents over-stretching; content centred

Test each breakpoint by: (1) Chrome DevTools device emulation, (2) physical device, (3) BrowserStack for real-browser verification on iOS.

6. Mobile Performance Testing

Metric	Target	How to Measure
Page load on 4G	< 3 seconds to interactive	Chrome DevTools → Network throttle → Fast 4G
Page load on 3G	< 6 seconds to interactive	Chrome DevTools → Network throttle → Fast 3G
Scroll performance	60fps smooth scroll; no jank	Chrome DevTools → Performance tab → record scroll

Metric	Target	How to Measure
Time to interactive	LCP < 2.5s on mobile	Lighthouse mobile audit
Memory usage	No steady heap growth over 30-min session	Chrome DevTools → Memory tab
Battery impact	App does not keep CPU busy when idle	Android Battery Historian; iOS Instruments
Image optimisation	Images use WebP/AVIF; responsive srcset served	Chrome DevTools → Network → filter Img; check Content-Type

7. iOS vs Android Behaviour Differences — Watchlist

Behaviour	iOS (Safari)	Android (Chrome)
Date picker	Native iOS date wheel — cannot style	Chrome uses native Android picker or HTML fallback
Input zoom	Auto-zooms on input focus if font-size < 16px	Does not auto-zoom; set font-size: 16px on inputs to prevent iOS zoom
Scroll behaviour	Momentum scrolling (inertia); -webkit-overflow-scrolling	Standard scroll; inertia less pronounced
Video autoplay	Blocked unless muted attribute present	Allowed with user gesture; muted autoplay permitted
PWA install	'Add to Home Screen' via Safari share sheet	Chrome install banner / ambient badge prompt
File input	'Choose File' opens iOS photo/file picker	Android file manager opens
Touch events	click fires ~300ms delayed; use touch events or FastClick	click fires with <50ms delay in Chrome
CSS -webkit prefixes	Many required for Safari	Generally not needed for modern Chrome

8. Mobile Bug Reporting — Extra Fields

Mobile defects require additional context beyond standard web bugs. Always include:

- Device make & model (e.g. iPhone 14 Pro, Samsung Galaxy S24)
- OS version (e.g. iOS 17.4.1, Android 14 — One UI 6.1)
- Browser name & version (e.g. Safari 17.4, Chrome 124)
- Network condition at time of test (WiFi, 4G, throttled 3G)
- Device orientation at time of issue (portrait/landscape)
- Screenshot from actual device (not just emulator)
- Video recording of the issue (BrowserStack session replay or screen record)
- Whether issue reproduces on emulator only or also on real device